自动驾驶决策规划算法第二章第五节

路径规划完结篇(提升篇)

二次规划崩溃问题

车控制不稳,抖动剧烈问题

决策不稳定,朝令夕改问题

速度规划如何影响路径规划

Q1: 在靠近障碍物时,二次规划崩溃

根源: 二次规划 $\frac{1}{2}x^T H x + f^T x = \min$

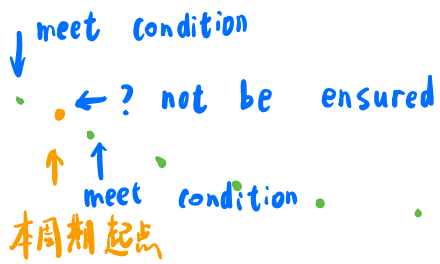
$Ax \le b$ 碰撞约束

$Aeq\,x = beq$ ⇓ 矛盾

$lb \le x \le ub$ 规划起点约束

规划起点: { 动力学递推 / 轨迹拼接 } 不能保证规划起点满足 $A_sub\,x \le b_sub$

上一周期

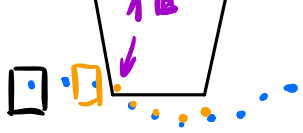


使用拼接时,规划起点约束与碰撞约束可能会发生矛盾

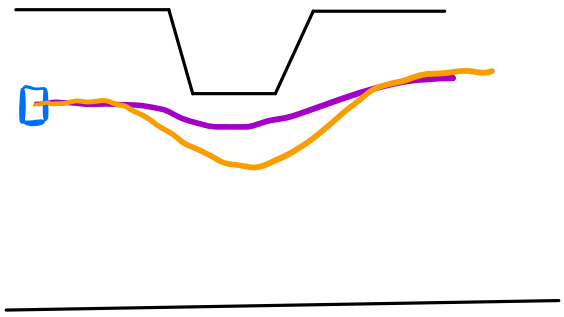
思考:为什么在无避障时不会崩溃



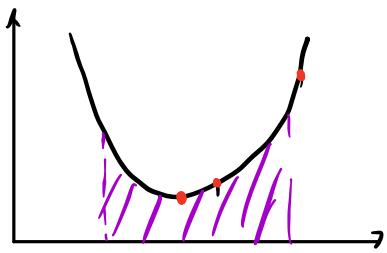




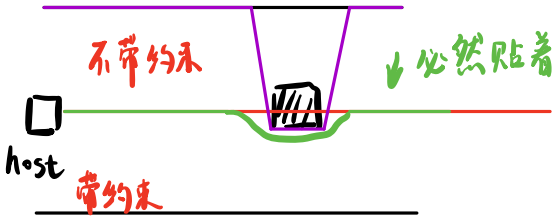
思考：为什么规划的路径会贴着边界



答：这是二次规划的性质决定的



带约束的二次规划的最小值只会在极小值点或边界点取得



$l^2 + l'^2 + l''^2 + l'''^2$

二次规划的曲线天然有往极小值靠拢的趋势，必然是贴着边界约束

改进：

① 规划起点不管了，障碍物放大一点做安全缓冲




```
for i = 1:n
    A ...
    b ...
end
```

=>

```
for i = 2:n
    A ...
    b ...
end
```

②改极小值点的位置

$H = l^2 + l'^2 + l''^2 + l'''^2 + (l - centre_line)^2$

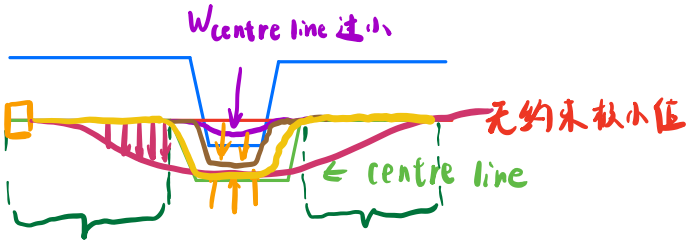
$w_l \cdot l^2 + \cdots + w_{centre} \cdot (l - centre_line)^2$ $w_{centre}\uparrow$ $w_l\downarrow$

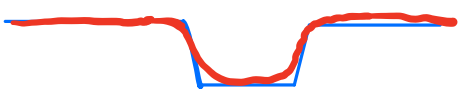


centre line $= \frac{1}{2}(l_{min} + l_{max})$

优点：几何稳定

缺点：w_{centre} 调小不起明显的作用，调大了路径不平滑

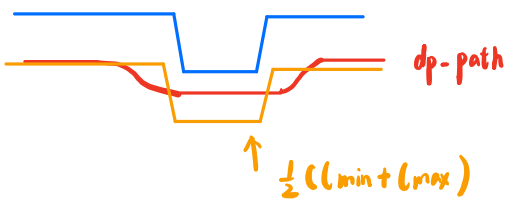




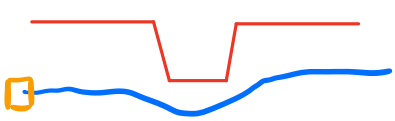
$w_{centre\ line}$ 过大又会导致曲线过于贴近 centre line，而 centre line 又不是一个平滑的 path

centre line = dp-path

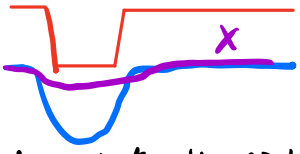
优点 dp-path 比 $\frac{1}{2}(l_{min} + l_{max})$ 平滑，曲线平滑相对好做



缺点：dp-path 几何不稳定



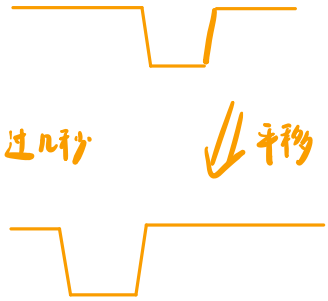
过了几秒



dp-path 每一帧之间的几何形状都会有缺变化，若二次规划以 dp-path 为 Centre line

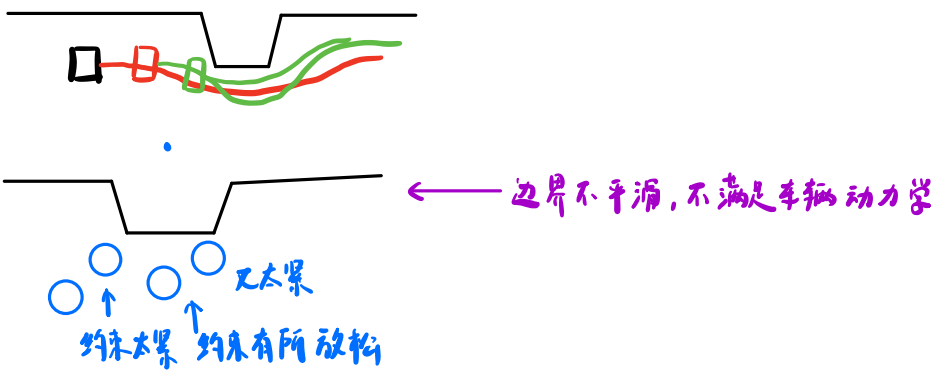
qp path 也会震荡

相反 centre line: $\frac{1}{2}(l_{min}+l_{max})$ 就很稳定



Q2: 方向盘大幅摆动，车抖的厉害

根源：车规划的路径会天然往边界靠拢的趋势




车在边界上当方向盘

解决：不允许反复横跳，加约束

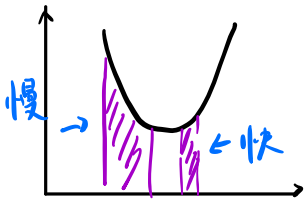
$$l'_{min} \le l'_i \le l'_{max}$$

$$l''_{min} \le l''_i \le l''_{max}$$

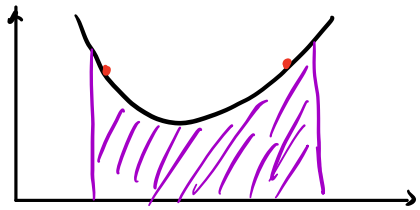
以及 $\le l'_{i+1} - l'_i \le$

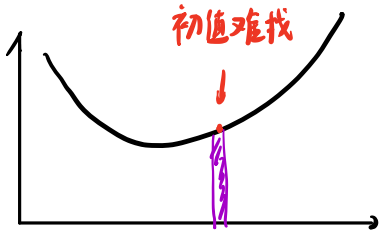
$\le l''_{i+1} - l''_i \le$

新的问题：约束越多，求解越慢（反直觉）



因为约束越多，初值越难找





解决：削二次规划的规模，由 60 → 20，二次规划求解完毕后在进行插值增密

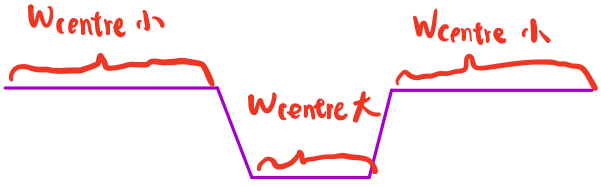
最终 Q_1、Q_2 合起来的改进方案

① 削 qp 规模，再插值增密

② 增加 l' l'' $l'_{in}-l_i$ $l''_{in}-l_i$ 的约束

③ 规划起点不做约束

④ 改 centre_line $=\frac{1}{2}(l_{min}+l_{max})$，合理分配权重



$f=-2\cdot W_{centre}\cdot centre_line$ 不好

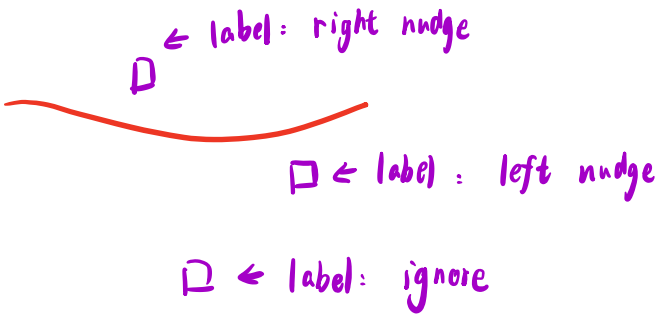
Q3：决策"朝令夕改"



解决：暂时无解 Matlab

Apollo，打标签

上一帧



这一帧：只对无标签的 obs 作动态规划决策，若障碍物已有标签，直接沿用标签的决策

Matlab 对结构体、字符串的支持有点弱，尽快切 C++

Q4: 速度规划对路径规划的影响

path → 只处理静态，有问题



此 case，速度规划无论怎么规划，都会碰撞（除非倒车）

怎么办？

已知上一帧的速度规划为 10m/s 匀速，上一帧的 path 为直线

这一帧



若 path 不考虑动态，则速度规划必无解

path 规划要拿上一帧的轨迹去判断与动态障碍物的交互

这一帧



上一帧的轨迹 这一帧的路径规划

上一帧的 path + speed planning 会影响到这一帧的 path planning

EM planner 是 SL planning 与 ST planning 反复迭代的过程

上一帧的 SL, ST —影响→ 当前 SL —影响→ 当前 ST —影响→ 下一帧 SL